

EXPERIMENT 29: BACKWARD CHAINING

Aim

To write a **Prolog program to implement Backward Chaining** by starting with a goal and working backward through rules to determine whether the goal can be proved.

Algorithm

1. Start.
2. Define facts and rules.
3. Specify a goal to be proved.
4. Search for a rule whose conclusion matches the goal.
5. Treat the rule's conditions as sub-goals.
6. Recursively prove each sub-goal.
7. If all sub-goals are satisfied, the goal is true.
8. Otherwise, backtrack and try another rule.
9. Display the result.
10. Stop.

Source Code

```
% Backward Chaining
```

```
fact(sunny).
```

```
fact(warm).
```

```
rule(pleasant, sunny).
```

```
rule(comfortable, warm).
```

```
rule(happy, pleasant).
```

```
rule(happy, comfortable).
```

```
prove(Goal) :-
```

```
fact(Goal).
```

```
prove(Goal) :-
```

```
    rule(Goal, Condition),
```

```
    prove(Condition).
```

Queries

```
?- prove(pleasant).
```

```
true.
```

```
?- prove(happy).
```

```
true.
```

```
?- prove(rainy).
```

```
false.
```

Output

```
?- [backward_chaining].
```

```
% backward_chaining.pl compiled 0.00 sec, 10 clauses
```

```
?- prove(pleasant).
```

```
true.
```

```
?- prove(happy).
```

```
true.
```

```
?- prove(rainy).
```

```
false.
```

```
?- 
```

Result

The Prolog program successfully implements **Backward Chaining** by starting with the required goal and recursively checking whether the necessary conditions can be proved from the available facts and rules.

